

GOnnect4

Rapport de projet

Version 1.0 du 12 janvier 2026

gonnect4.ch
github.com/marvinEgger/GOnnect4

Astrit Aslani, Marvin Egger



Haute école d'ingénierie et d'architecture Fribourg
Hochschule für Technik und Architektur Freiburg

Table des matières

1 Introduction	1
2 Architecture réseau client-serveur	1
2.1 Communication WebSocket	1
2.2 Gestion des Sessions et Reconnexion	2
2.3 Thread-Safe et Concurrency	2
3 Architecture des données Graphe de Nodes	3
3.1 Structure de Graphe avec 8 Voisins	3
3.2 Détection de Victoire en $O(1)$	3
4 Gestion des parties et leurs cycles de vies	4
4.1 États d'une partie	4
4.2 Matchmaking et Parties Privées	4
5 Client WebAssembly	5
5.1 State Management Centralisé	5
5.2 Synchronisation en temps réel	5
5.3 Gestion de la Mémoire WASM	5
6 Pipeline CI/CD et déploiement automatisé	6
6.1 Architecture de production	6
6.2 Workflow CI/CD avec GitHub Actions	6
6.3 Processus de déploiement et configuration	6
7 Conclusion	7
7.1 Astrit	7
7.2 Marvin	7
8 Table des figures	7

1 Introduction

Ce rapport présente le développement de GOnnect4, une implémentation complète du jeu « Connect 4 » permettant à des joueurs de s'affronter en temps réel via une architecture client-serveur. Le projet met en œuvre une communication WebSocket bidirectionnelle, un système de sessions persistantes avec reconnexion automatique, ainsi qu'un algorithme de détection de victoire optimisé grâce à une structure de graphe.

2 Architecture réseau client-serveur

L'architecture repose sur un modèle client-serveur classique où le serveur Go maintient l'état autoritaire de toutes les parties en cours et les clients WebAssembly envoient leurs actions via WebSocket. Cette approche garantit que toute la logique métier reste centralisé côté serveur, empêchant ainsi toute tentative de triche et assurant la cohérence entre tous les joueurs.

2.1 Communication WebSocket

Le protocole WebSocket a été retenu pour établir une connexion bidirectionnelle persistante entre chaque client et le serveur. Contrairement à une approche HTTP classique basée sur des requêtes-réponses, WebSocket permet au serveur de pousser des événements vers les clients dès qu'un changement d'état se produit, garantissant ainsi une synchronisation en temps réel.

L'ensemble de la communication repose sur un format de messages JSON structurés. Chaque message possède un champ type identifiant l'action ou l'événement, ainsi qu'un champ data contenant les informations spécifiques au contexte. Cette structure standardisée facilite l'extension future du protocole et permet d'ajouter de nouveaux types de messages sans modifier l'architecture existante.

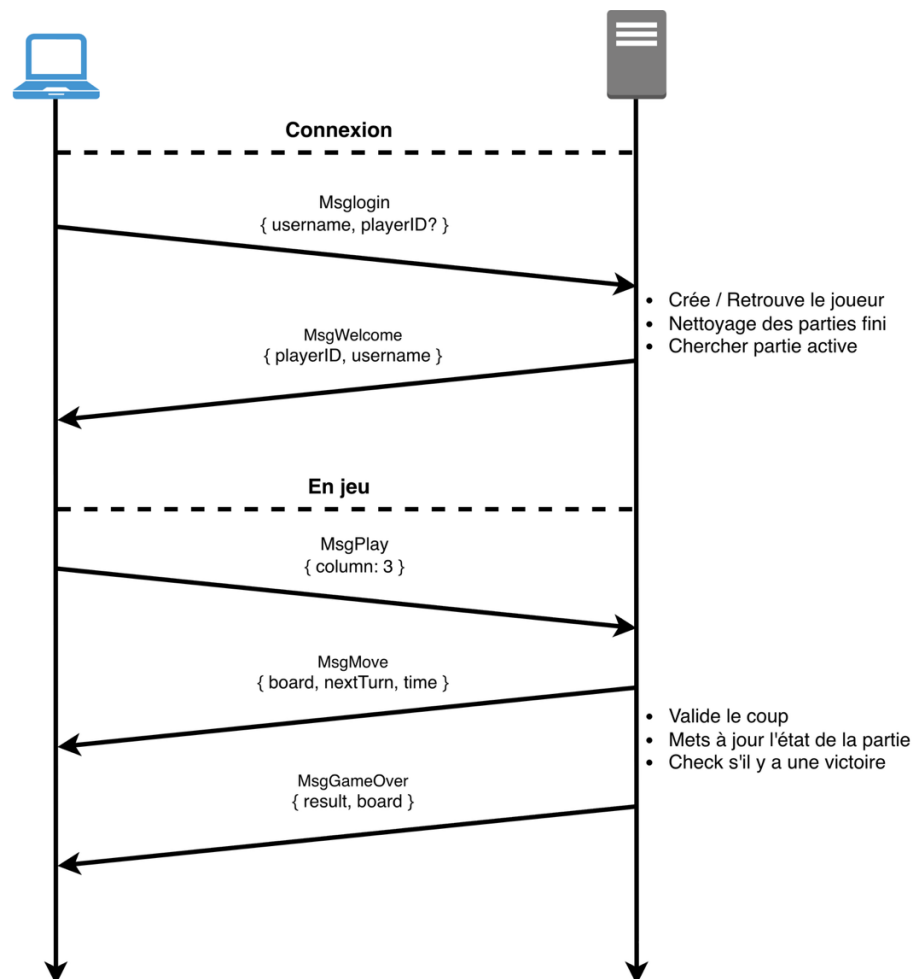


Figure 1 : Flow de communication WebSocket entre client et serveur

2.2 Gestion des Sessions et Reconnexion

Chaque joueur reçoit un identifiant unique (`PlayerID`) lors de sa première connexion. Cet identifiant est stocké en `localStorage` côté client et transmis lors des reconnections ultérieures. Cette approche permet au système de distinguer une nouvelle connexion d'une tentative de reconnexion, et d'adapter le comportement.

Lorsqu'un joueur se reconnecte, le serveur vérifie l'existence de son `PlayerID` dans le lobby. Si le joueur est retrouvé, le serveur effectue un nettoyage des parties terminées dans lesquelles le joueur pourrait encore être enregistré, puis recherche une partie active en cours. Dans le cas où une partie en attente ou en cours est trouvée, le serveur renvoie l'état complet de cette partie au client, permettant une reprise transparente.

Cette logique de reconnexion résout un problème critique identifié lors de nos tests, les joueurs peuvent rester bloqués dans d'anciennes parties terminées, les empêchant de rejoindre de nouvelles parties ou de jouer. Le nettoyage systématique des parties finies avant toute action (création de partie, matchmaking, reconnexion) garantit que les joueurs ne sont jamais associés à plusieurs parties simultanément.

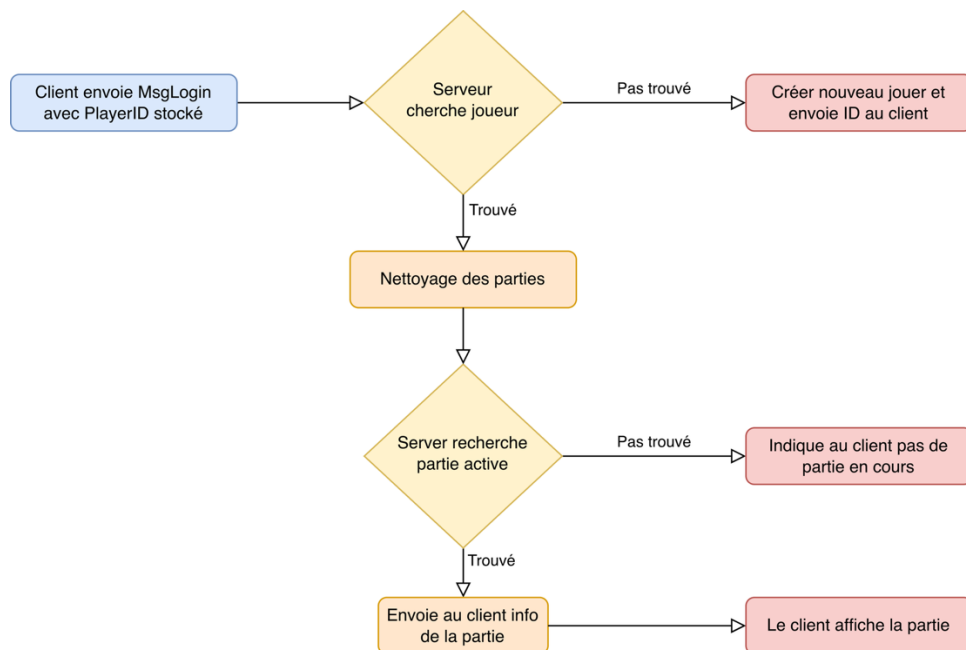


Figure 2 : Flowchart de la gestion de reconnections

`cleanupPlayerFinishedGames()` parcourt toutes les parties terminées et retire le joueur en appelant `game.RemovePlayer()`, une méthode thread-safe qui modifie directement le tableau `Players` de la partie.

2.3 Thread-Safe et Concurrency

Le serveur gère simultanément plusieurs parties et connexions WebSocket. Toutes les structures critiques (`Game`, `Player`, `Server`) utilisent `sync.RWMutex` pour protéger les accès concurrents. Les opérations de lecture utilisent `RLock` / `RUnlock` permettant plusieurs lectures simultanées, tandis que les modifications utilisent `Lock` / `Unlock` pour garantir l'exclusivité. Exemple avec des extraits du `server/lib/game.go` :

Lecture :

```
// GetPlayerIndex returns the index of player
func (g *Game) GetPlayerIndex(id PlayerID) int
{
    g.mu.RLock()
    defer g.mu.RUnlock()
    return g.getPlayerIndexLocked(id)
}
```

Écriture :

```
// Play attempts to play a move in the a column
func (g *Game) Play(playerIdx, col int) error {
    g.mu.Lock()
    defer g.mu.Unlock()
    // ...
}
```

Cette approche évite les race conditions, notamment sur le champ `GameCode`.

3 Architecture des données Graphe de Nodes

L'une des particularités techniques du projet réside dans la représentation du plateau de jeu sous forme de graphe plutôt que de simple matrice bidimensionnelle. Cette approche permet d'optimiser drastiquement la détection de victoire en réduisant la complexité en $O(1)$ par coup joué.

3.1 Structure de Graphe avec 8 Voisins

Au lieu de représenter le plateau comme un tableau 2D classique, chaque cellule est modélisée comme un Node contenant des pointeurs vers ses huit voisins directs : haut, bas, gauche, droite, et les quatre diagonales. Lors de l'initialisation du plateau, tous ces liens sont établis une seule fois, créant ainsi un graphe complet et navigable.

```
// Node represents a cell in the graph node board
type Node struct {
    Row      int
    Col      int
    // CellEmpty, CellPlayer0, CellPlayer1
    Owner    Cell
    // 8 voisins (nil aux bords)
    Neighbors [dirCount]*Node
}
```

Cette structure permet de naviguer dans le graphe en temps constant. Pour obtenir le voisin dans une direction donnée (représentée par un index 0-7), il suffit de suivre le pointeur correspondant sans avoir à calculer de coordonnées ni à vérifier les limites du plateau.

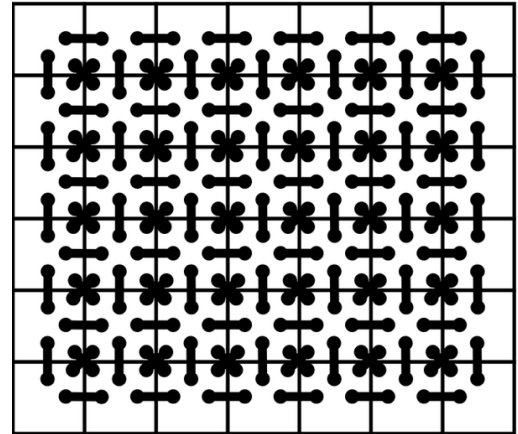


Figure 3 : Représentation de la structure à 8 voisins

3.2 Détection de Victoire en $O(1)$

L'algorithme de détection exploite cette structure pour ne vérifier que les alignements incluant le coup qui vient d'être joué. Au lieu de parcourir tout le plateau à chaque coup, l'algorithme explore dans quatre directions principales (verticales, horizontales et diagonales) en comptant les nœuds alignés dans la direction.

Le code utilise une méthode `countSequence` qui parcourt les voisins dans une direction donnée :

```
// countSequence counts nodes in given direction
func (n *Node) countSequence(dir Direction) int {
    if n.IsEmpty() {
        return 0
    }

    count := 0
    current := n.GetNeighbor(dir)

    for current != nil && current.Owner == n.Owner {
        count++
        current = current.GetNeighbor(dir)
    }

    return count
}
```

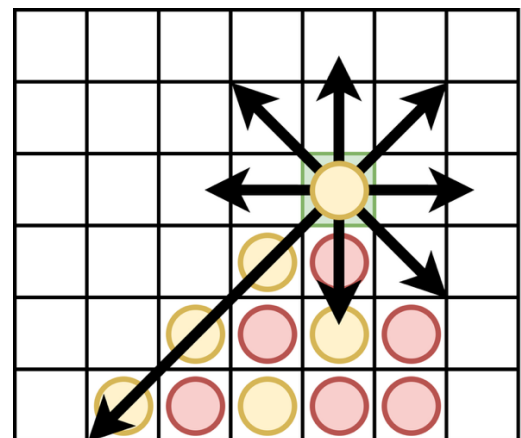


Figure 4 : Directions vérifiées pour une victoire

Grâce à la structure de graphe, cette exploration ne nécessite aucun calcul d'indice ni vérification de limites. La complexité reste constante car dans le pire cas, il suffit de vérifier au maximum les 8×4 cellules = 32 comparaisons, indépendamment de la taille du plateau.

Cette approche est significativement plus efficace qu'un scan complet du plateau après chaque coup, qui nécessiterait de parcourir l'ensemble des 42 cellules pour détecter tous les alignements possibles.

4 Gestion des parties et leurs cycles de vies

Le serveur maintient deux structures principales pour gérer les joueurs et les parties. Un `lobby` contenant tous les joueurs connectés, et `gamesByCode` regroupant toutes les parties actives indexées par leur code unique. Cette séparation permet de gérer indépendamment le cycle de vie des joueurs et celui des parties.

4.1 États d'une partie

Une partie traverse plusieurs états au cours de son existence, matérialisés par l'énumération `GameStatus`. L'état `StatusWaiting` correspond à une partie créée mais en attente du second joueur. Dès que deux joueurs sont présents, la partie passe à `StatusPlaying` et les chronomètres démarrent. Lorsqu'un joueur gagne, abandonne ou dépasse son temps, la partie passe à `StatusFinished`. Enfin, l'état `StatusCleaned` indique qu'une partie a été nettoyée et ne doit plus être manipulée.

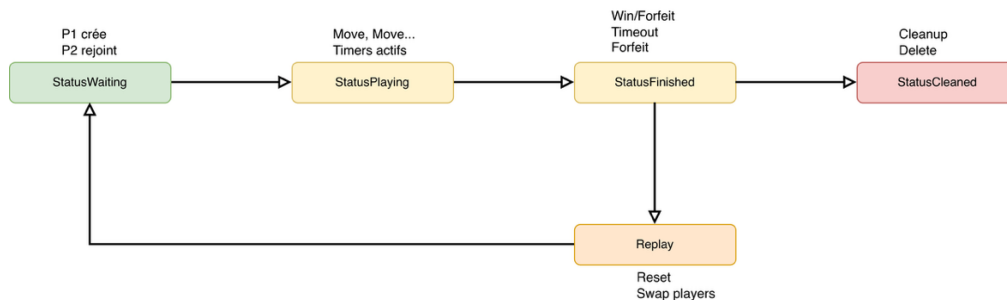


Figure 5 : Diagramme d'états d'une partie

La transition entre ces états est gérée de manière thread-safe grâce aux mutex de la structure `Game`. Chaque modification d'état s'accompagne d'actions spécifiques, comme le démarrage des chronomètres lors du passage à `StatusPlaying`, arrêt des chronos lors du passage à `StatusFinished` et libération des ressources lors du passage à `StatusCleaned`.

4.2 Matchmaking et Parties Privées

Le système propose deux modes de jeu distincts. Le mode partie privée permet à un joueur de créer une partie avec un code unique qu'il peut partager avec un ami. Le second joueur rejoint simplement en entrant le code. Le mode lui matchmaking automatique met en relation deux joueurs en attente via une file d'attente.

Lorsqu'un joueur rejoint la file, le serveur vérifie s'il y a au moins deux joueurs en attente. Si c'est le cas, les deux premiers joueurs sont associés et retirés de la queue (extrait de `server/matchmaking.go`) :

```

func (srv *Server) tryMatchPlayers() {
    // Take first two players
    player1ID := srv.matchmakingQueue[0]
    player2ID := srv.matchmakingQueue[1]
    srv.matchmakingQueue = srv.matchmakingQueue[2:] // Remove from queue

    ...

    // Create game
    game := lib.NewGame(initialClockDuration)
    game.TimerCallback = srv.handleTimeout
    game.AddPlayer(player1)
    game.AddPlayer(player2)
    srv.gamesByCode[game.Code] = game

    // Notify both players
    srv.broadcastToGame(game, lib.Message{ ... })
}
  
```

Cette approche connecte deux joueurs simplement (premier arrivé, premier servi). Le serveur diffuse aussi le nombre de joueurs en attente à pour l'affichage en temps réel dans l'interface.

5 Client WebAssembly

Le client est compilé depuis Go vers WebAssembly et s'exécute nativement dans le navigateur. Cette approche permet de partager le code métier (structures de données, messages, validation) entre le serveur et le client, tout en bénéficiant des performances natives de WASM.

5.1 State Management Centralisé

L'architecture client repose sur un state management centralisé utilisant un pattern de Singleton. Une unique structure State contient l'ensemble de l'état de l'application (plateau de jeu, joueurs, temps restant, tour actuel) et protège tous les accès avec `sync.RWMutex`. Tous les composants accèdent à cet état via des getters et setters thread-safe (`GetBoard()`, `SetCurrentTurn()`, etc.), garantissant la cohérence avec les goroutines WASM et les callbacks.

5.2 Synchronisation en temps réel

Le client maintient une représentation locale de l'état du jeu et se synchronise avec le serveur via WebSocket. À chaque coup joué, le serveur renvoie l'état complet (plateau, tour suivant, temps restant) que le client applique localement. Un chrono local décrémente le chronomètre du joueur actif toutes les 100ms et applique des classes CSS (warning à 30s, danger à 10s) pour un feedback visuel immédiat.

5.3 Gestion de la Mémoire WASM

WebAssembly impose une gestion explicite de la mémoire pour les callbacks JavaScript. Les fonctions Go exposées au JavaScript (`js.FuncOf`) créent des références qui doivent être explicitement libérées avec `Release()` pour éviter les fuites mémoire. Le client implémente une structure `wsCallbackFuncs` qui regroupe tous les callbacks WebSocket (`onOpen`, `onMessage`, `onError`, `onClose`) et les libère systématiquement avant chaque reconnexion.

```
// Connect establishes WebSocket connection
func Connect(username, playerID string, onMessage func(Message)) {
    messageHandler = onMessage

    // Release previous callbacks if any
    if wsCallbacks != nil {
        wsCallbacks.onOpen.Release()
        wsCallbacks.onMessage.Release()
        wsCallbacks.onError.Release()
        wsCallbacks.onClose.Release()
    }

    // Create new callbacks
    ...

    // Register handlers BEFORE connection opens
    ws.Call("addEventListener", "error", wsCallbacks.onError)
    ws.Call("addEventListener", "close", wsCallbacks.onClose)
    ws.Call("addEventListener", "open", wsCallbacks.onOpen)
    ws.Call("addEventListener", "message", wsCallbacks.onMessage)
}
```

Cette gestion résout les fuites mémoire identifiées en production après plusieurs reconnexions successives.

En résumé, le client WASM fait exactement ce qu'on ferait en JavaScript classique (state management, affichage DOM, timers, WebSocket), mais en Go avec des performances natives et un partage de code avec le serveur.

6 Pipeline CI/CD et déploiement automatisé

Le projet s'accompagne d'une infrastructure complète de déploiement automatisé permettant de passer du code source à une application en production en quelques minutes, sans intervention manuelle.

6.1 Architecture de production

L'architecture de production repose sur une stack combinant Nginx et Docker. Le serveur Go s'exécute dans un container Docker isolé, écoutant uniquement sur `localhost:8080`. Nginx agit comme reverse proxy, gérant le HTTPS et servant les fichiers statiques directement.

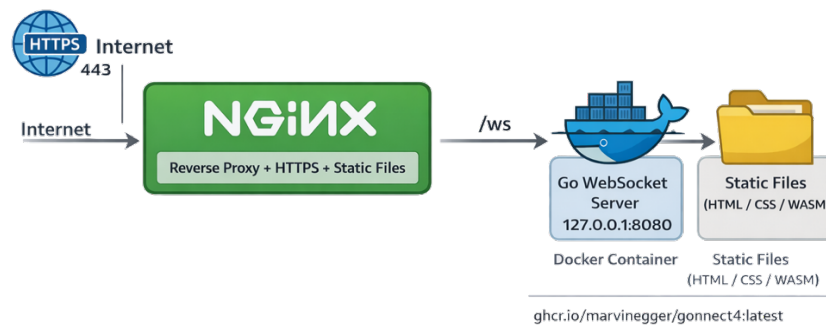
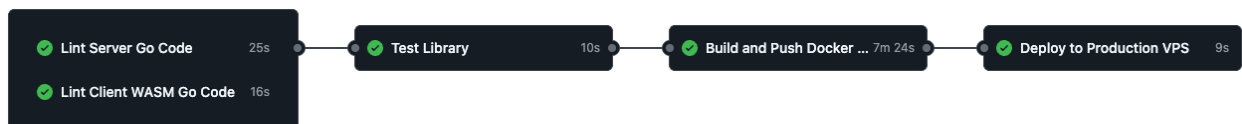


Figure 6 : Architecture de déploiement en production (Généré avec ChatGPT)

6.2 Workflow CI/CD avec GitHub Actions

La pipeline CI/CD s'appuie sur les GitHub Actions et se déclenche selon deux modes distincts. Lors d'un push sur main, seuls les jobs de tests et de linting s'exécutent, fournissant un feedback rapide. Lors d'un push de tag de version (par exemple 1.0.3), l'ensemble du workflow s'exécute avec le linting, tests unitaire construction de l'image Docker, push sur GitHub Container Registry, et déploiement automatique sur le VPS.



Cette approche permet un développement itératif rapide (push sur main → feedback en quelques minutes) tout en gardant le contrôle sur les déploiements en production (création explicite d'un tag → déploiement automatique). Le versioning sémantique devient ainsi le mécanisme de déclenchement des releases.

6.3 Processus de déploiement et configuration

Le déploiement s'exécute via SSH sur le VPS et enchaîne plusieurs opérations. L'image Docker est téléchargée depuis GitHub Container Registry, puis le container redémarre via `docker compose up -d`. Les fichiers client (HTML, CSS, JS, WASM) sont extraits dans un container temporaire et copiés vers `/opt/GOnnect4/client/` pour être servis par Nginx. Enfin, Nginx recharge sa configuration.

Étapes du déploiement automatique :

- | | |
|---|---|
| 1. <code>docker compose pull</code> | → Télécharge <code>ghcr.io/.../gonnect4:latest</code> |
| 2. <code>docker compose up -d</code> | → Redémarre le container (rolling restart) |
| 3. <code>docker create temp</code> | → Container temporaire pour extraction |
| 4. <code>docker cp temp:/app/client/</code> | → Copie fichiers WASM/HTML/CSS/JS |
| 5. <code>docker rm temp</code> | → Supprime container temporaire |
| 6. <code>systemctl reload nginx</code> | → Recharge config Nginx |

Cette technique garantit que Nginx sert toujours les fichiers correspondant exactement à la version du serveur déployée, évitant toute désynchronisation entre client et serveur. Le déploiement nécessite trois secrets GitHub : `SSH_HOST` (IP du VPS), `SSH_USER` (nom d'utilisateur), et `SSH_KEY` (clé privée SSH).

7 Conclusion

GOnnect4 illustre la puissance et l'élégance de Go pour développer des applications temps réel complètes. Nous avons fait le choix d'abandonner Ebitengine initialement envisagé pour adopter une architecture client-serveur web, permettant d'utiliser du simple HTML / CSS pour l'interface tout en conservant Go côté client via WebAssembly. Cette décision nous a offert beaucoup plus de liberté et de contrôle pour obtenir exactement le résultat que nous imaginions, tout en restant pleinement dans le cadre du cours de programmation élégante en Go.

L'architecture client-serveur nous a permis de tirer parti des forces de Go, avec une gestion native de la concurrence avec les goroutines et les mutex, la compilation vers WebAssembly pour partager du code entre client et serveur, et la simplicité du langage qui rend le code lisible et maintenable. Le résultat est une application stable déployée en production disponible sur gonnect4.ch, démontrant qu'on peut créer une expérience web moderne entièrement en Go.

7.1 Astrit

Le fait d'avoir bien défini l'architecture au départ nous a permis de travailler en parallèle efficacement, et c'est toujours un plaisir de collaborer avec Marvin, on a une bonne entente et une vision similaire du code.

Go était vraiment le langage parfait pour ce projet. La gestion native de la concurrence avec les goroutines nous a permis de gérer facilement les parties WebSocket en parallèle sans se prendre la tête avec des threads ou des callbacks. Le fait de pouvoir compiler vers WASM est top, on a pu écrire tout notre code en Go (client et serveur), partager les structures de données et les messages, et éviter complètement JavaScript. Ça nous a donné une cohérence totale. Et la simplicité du langage rend le code vraiment lisible, on peut revenir dessus des semaines après et comprendre directement ce qui se passe.

7.2 Marvin

J'ai trouvé ce projet était une bonne expérience et très intéressant à réaliser. Nous avons consacré pas mal de temps au début pour définir comment nous allions réaliser le projet : la structure, le fonctionnement du jeu et les fonctionnalités que nous allions implémenter.

Cela nous a permis d'avancer chacun de notre côté pendant l'implémentation, puis de mettre régulièrement notre travail en commun sans difficulté. Je suis content du résultat final, je pense que notre code est propre et que nous nous sommes bien organisés.

8 Table des figures

Tous les fichiers draw.io utilisées pour la création des figures sont disponibles dans le repository GitHub.

Figure 1 : Flow de communication WebSocket entre client et serveur	1
Figure 2 : Flowchart de la gestion de reconnections.....	2
Figure 3 : Représentation de la structure à 8 voisins	3
Figure 4 : Directions vérifiées pour une victoire	3
Figure 5 : Diagramme d'états d'une partie.....	4
Figure 6 : Architecture de déploiement en production (Généré avec ChatGPT)	6